RAGS: Reinforcement Learning-based Global Scheduling for LLM Agent Serving Systems

Anonymous Author(s)

Abstract

LLM-based Agent systems compose multiple inference calls into dynamic execution graphs, posing three fundamental challenges for global scheduling: nonlinear topologies with conditional branches and feedback loops, heterogeneous per-node inference costs, and KV cache affinity in tension with load balance. Existing schedulers address these only partially—topology-aware queuing breaks down when dynamic branches invalidate static stage estimates, while Round-Robin and Session Affinity represent opposite extremes that trade off cache reuse against load balance without adaptation. We present **RAGS** (RL-based Agent Global Scheduler), a two-tier framework that co-designs queuing and dispatch to address these challenges jointly. The queuing tier, **DynA**, prioritizes requests via an online-estimated conditional near-completion probability over per-Agent-type completion-round distributions, augmented with cumulative-latency-aware ordering and anti-starvation, all without relying on any predefined topology. The dispatch tier models request-to-instance assignment as an MDP and trains a Transformer-based policy with Double-DQN to jointly optimize KV cache reuse, load balance, and queue congestion via offline pretraining and online fine-tuning. Experiments on a simulated 4-instance LLM cluster across three representative Agent workloads show that RAGS reduces mean end-to-end latency by up to **5.2**× over Round-Robin and **3–4**× over Parrot and Ayo; ablation and heterogeneous GPU cluster experiments further confirm the complementary contributions and generalizability of both tiers.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

LLM serving, Agent workflows, global scheduling, reinforcement learning, KV cache, load balancing, queuing algorithms

ACM Reference Format:
Anonymous Author(s). 2018. RAGS: Reinforcement Learning-based Global Scheduling for LLM Agent Serving Systems. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large Language Models (LLMs) have rapidly advanced from standalone research artifacts to the computational backbone of a broad class of intelligent applications. Modern LLM-based Agent systems—encompassing multi-turn chatbots, autonomous code generation pipelines, and large-scale document processing workflows—increasingly compose multiple LLM inference calls into structured, multi-step execution graphs. As such Agent deployments scale from research prototypes to production services handling thousands of concurrent workflows, the efficiency of the underlying LLM inference infrastructure has become a critical concern. In this context, the *global scheduler* of a multi-tenant LLM serving cluster plays a central role: it must simultaneously determine the *execution order* of pending requests and *dispatch* each request to an appropriate backend instance, with the overarching goal of minimizing end-to-end workflow latency under dynamic, heterogeneous load.

Classical scheduling strategies developed for stateless, single-turn inference fail to meet this challenge because they rest on assumptions that LLM Agent workloads systematically violate. First, Agent workflows exhibit *dynamic, nonlinear topologies*—including conditional branches and iterative repair loops—whose actual execution paths and depths are only determined at runtime, precluding static graph-based priority estimation. Second, requests emitted at different stages of the same workflow are *highly heterogeneous* in input length, output length, and GPU memory footprint, so uniform treatment causes severe head-of-line blocking and resource imbalance. Third, adjacent calls within a workflow share heavily overlapping context prefixes, creating significant *KV cache reuse* opportunities that are tightly coupled with, and in tension with, global load balance.

Existing approaches address these challenges only partially. On the queuing side, First-Come-First-Served (FCFS) is entirely blind to workflow progress, leading to cascading latency amplification along data-flow edges. Topology-aware strategies such as Ayo [22] assign priority by static residual stage counts, but break down when dynamic branches and loops cause the estimated stage depth to diverge sharply from the true execution trajectory. On the dispatch side, Round-Robin distributes requests evenly without exploiting prefix-cache locality, while Session Affinity maximizes cache reuse but creates severe hotspots when parallel fan-out stages concentrate bursty requests on a single instance. Parrot [14] employs static session binding and multi-objective linear weighting, yet its fixed-weight heuristics cannot adapt to the continuously shifting cluster state induced by dynamic Agent workloads, causing catastrophic load imbalance under feedback-repair scenarios. Taken together, these limitations demonstrate that effective global scheduling for

LLM Agent serving requires the *co-design* of queuing and dispatch: neither tier can achieve robust performance in isolation.

We present **RAGS** (RL-based Agent Global Scheduler), a two-tier global scheduling framework that addresses both dimensions in a coordinated manner. The queuing tier, **DynA**, makes all priority decisions using only runtime-observable signals—online-estimated completion-round distributions, accumulated workflow latency, and per-request queue age—without any dependence on a predefined workflow topology. The dispatch tier employs a *reinforcement learning* policy that models request-to-instance assignment as a Markov Decision Process and trains a Transformer-based network with Double-DQN to jointly optimize KV cache reuse, load balance, and queue congestion, adapting continuously through offline pretraining and online fine-tuning.

The main contributions of this paper are as follows.

- We propose a dynamic-aware queuing algorithm that estimates each request's near-completion probability from online-collected statistics, enabling workflow-progress-aware priority ordering and reducing head-of-line blocking across concurrent workflows.
- To the best of our knowledge, RAGS is the first system to apply reinforcement learning to global dispatch in LLM Agent serving. By jointly encoding heterogeneous cluster features—including KV-cache state, load histograms, and GPU hardware type—the RL policy achieves adaptive multi-objective optimization that consistently outperforms static rule-based policies.
- Evaluation on a simulated LLM system shows that RAGS reduces mean end-to-end latency by up to **5.2**× over Round-Robin and **3–4**× over Parrot and Ayo. Ablation studies and heterogeneous GPU cluster experiments further confirm the complementary contributions and generalizability of both tiers.

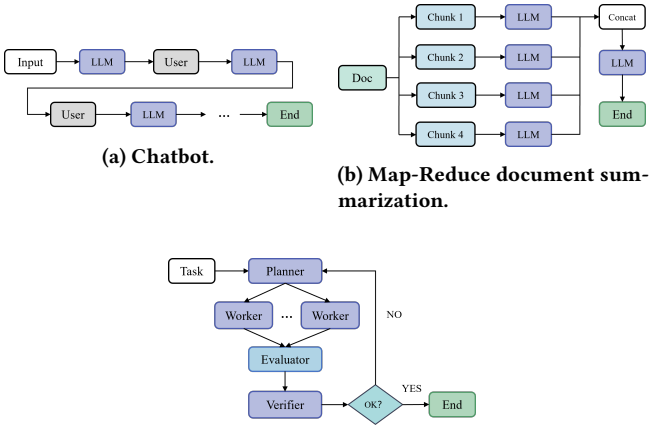
2 Motivation

LLM serving systems typically adopt a layered scheduling architecture in which a *global scheduler* is responsible for two coupled decisions: (1) determining the *execution order* of pending requests (queuing policy), and (2) assigning each request to a target backend instance (dispatch policy). This section characterizes the distinctive properties of LLM Agent workloads and demonstrates why standard request-level heuristics for both decisions are fundamentally inadequate, motivating the co-design presented in Section 3.

2.1 Characteristics of LLM Agent Workflows

To gain a comprehensive understanding of LLM Agent characteristics, we surveyed mainstream open-source Agent projects on GitHub [3, 9, 17] and systematically analyzed representative workflow paradigms, including chatbots, code generation, and document summarization. As illustrated in Figure 1, typical Agent workflows can be abstracted as directed graphs in which each node represents a single LLM inference call or tool invocation, and each edge encodes a data-flow or control dependency. Based on this survey, Agent workloads at the system level exhibit three distinctive traits.

Dynamic, nonlinear topology with mixed concurrency. Our analysis reveals that Agent workflows deviate substantially from



(a) Chatbot.

(b) Map-Reduce document summarization.

(c) Code generation.

Figure 1: Representative LLM Agent workflow structures.

the static Directed Acyclic Graph (DAG) assumptions typical of traditional microservices. They inherently incorporate conditional branches and feedback loops (e.g., iterative repair cycles triggered by validation failures), alongside an indeterminate number of user-driven interaction rounds. Furthermore, these workflows frequently interleave strictly sequential, blocking stages with highly parallel fan-out operations (e.g., the Map phase in document summarization). Consequently, this combination of severe dynamism and irregular concurrency renders end-to-end execution paths highly unpredictable, posing formidable challenges for global resource planning.

Highly heterogeneous per-node inference cost. Because nodes serve different functional roles—routing, generation, quality review, etc.—the LLM requests they emit differ substantially in input prompt length, output length, and end-to-end execution latency. This heterogeneity is confirmed by prior work reporting that latency differences across roles within a single application can exceed 25× [6], requiring fine-grained, cost-aware resource management from the scheduler.

Deep context sharing and KV cache affinity. Agent requests typically carry a fixed system prompt alongside an ever-growing conversation history. Across rounds of the same workflow, the system prompt remains constant and adjacent calls share heavily overlapping context prefixes, creating substantial opportunities for KV cache reuse. However, affinity-driven routing that always steers requests to the same instance tends to concentrate load and produce hotspots. Balancing KV cache hit rate against cluster-wide load balance is therefore a central tension in global scheduling.

Taken together, these traits violate the classical assumption of independent, homogeneous requests. The following two subsections examine the concrete limitations that arise when standard queuing and dispatch policies are applied to Agent workloads.

2.2 Limitations of Existing Queuing Algorithms

Conventional LLM serving paradigms model requests as independent, single-turn calls, where standard First-Come-First-Served (FCFS) queuing suffices to maintain fairness. Agent workloads are

inherently interdependent, however, introducing three fundamental scheduling complexities.

(1) Head-of-line blocking due to workflow-progress heterogeneity. Concurrent Agent requests often exhibit substantial heterogeneity in their execution progress, i.e., at any given moment, the system processes a mixture of initial LLM calls and near-terminal tasks (Section 2.1). Being completely progress-agnostic, FCFS inevitably stalls near-completion requests behind newly admitted early-stage tasks, thereby artificially inflating end-to-end workflow latencies.

(2) Cascading amplification and accumulation of queuing delays. Unlike isolated single-turn requests, queuing delays experienced at any individual workflow node propagate along dataflow dependencies. This cascading effect inherently stalls downstream execution. Moreover, since FCFS ranks requests purely by arrival time, accumulated workflow-level latency is entirely ignored, causing tail performance to degrade as delays compound across successive stages.

(3) Inability to handle dynamic workflow topologies. Systems such as Ayo [22] address priority assignment with a *workflow-topology-aware* strategy (Topo) that assigns higher priority to tasks with fewer remaining stages. However, Topo requires a complete, static workflow graph; when dynamic branches or loops fire at runtime (Section 2.1), the statically estimated residual stage counts diverge sharply from true values, causing tasks in actively expanding phases to be systematically under-prioritized.

Agent	Stages	Cost
A	1	{10}
B	2	{4, 4}
C	3	{1, 1, 1}
D	1	{1}

Policy	Makespan
FCFS	10+10+11+6=37
Topo	10+9+11+1=31
Oracle	15+7+3+1=26

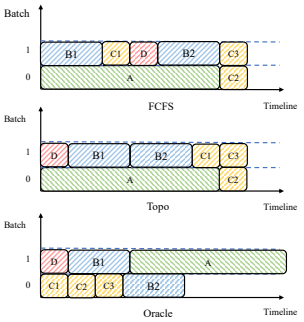


Figure 2: End-to-end latency comparison under FCFS, topology-aware (Topo), and globally E2E-optimal queuing policies across four concurrent Agent tasks.

Figure 2 illustrates these effects with four concurrent Agent tasks on a single LLM instance (maximum batch size 2; per-task stage counts and durations are provided in the figure). Under FCFS, all four jobs' initial requests arrive nearly simultaneously, so the scheduler interleaves them indiscriminately, causing severe blocking and an average completion time of $(10 + 10 + 11 + 6)/4 = 9.25$ s. Topo assigns priority by static stage count and preferentially schedules tasks with fewer remaining stages (e.g., tasks A and D), reducing the average to $(10+9+11+1)/4 = 7.75$ s. An *E2E-cost-aware* policy that globally accounts for each task's total remaining work can further lower the average completion time to $(15 + 7 + 3 + 1)/4 = 6.5$ s, demonstrating the headroom available to a more informed scheduling strategy.

2.3 Limitations of Existing Dispatch Strategies

Classical global dispatchers for LLMs rely predominantly on Round-Robin (RR), which distributes requests across instances in sequence to achieve load balance, or Session Affinity (SA), which routes all requests from the same session to a fixed instance to maximize KV cache reuse. Both strategies expose pronounced weaknesses when confronted with Agent workloads.

RR lacks awareness of request heterogeneity. Requests from different functional nodes vary greatly in context length and GPU memory footprint. RR's uniform assignment can inadvertently concentrate multiple memory-intensive requests on the same instance, saturating its GPU memory and triggering preemption. Preempted requests not only waste the computation already performed but also incur additional latency from re-queuing and recomputation, reducing system throughput.

SA ignores bursty concurrency within a workflow. In a Map–Reduce-style document summarization Agent, the Map phase emits dozens of parallel subtask requests simultaneously. SA binds all of them to a single instance, creating a severe local hotspot while other instances remain idle—fundamentally undermining the goal of cluster-wide dynamic load balance.

To illustrate these trade-offs concretely, Figure 3 compares RR and SA on two contrasting workflow topologies, using a setup of 4 homogeneous LLM instances (batch size 1 each), a base inference latency of 100 ms, and 80 ms when the system-prompt KV cache is hit. For an 8-subtask *parallel* (Map-phase) workflow, SA forces all concurrent requests onto one instance, yielding $100+7\times80 = 660$ ms, whereas RR distributes them for true parallelism, achieving only $100 + 80 = 180$ ms. For an 8-subtask *sequential* (chain) workflow, the positions are reversed: RR routes each request to a different instance, destroying prefix-cache locality and inflating latency to $4\times100 + 4\times80 = 720$ ms, while SA retains cache reuse and holds latency at $100 + 7\times80 = 660$ ms. Neither policy dominates; the optimal choice depends jointly on execution topology, inter-request temporal dependencies, and live cache state.

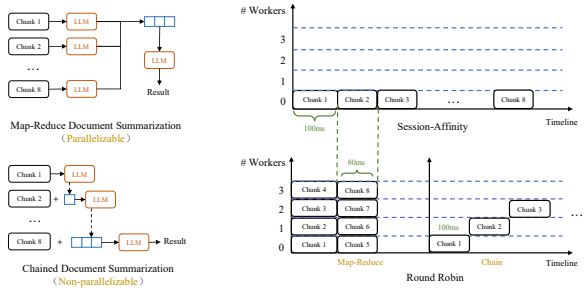


Figure 3: RR versus Session Affinity under parallel and sequential document-summarization Agent workflows.

To address these limitations, recent systems incorporate richer heuristics such as multi-objective linear weighting [21]. However, LLM cluster state evolves continuously: request arrival patterns, per-instance queue lengths, and GPU memory utilization all change

over time, and each dispatch decision reshapes the state distribution encountered by subsequent requests. Fixed-weight heuristics are ill-equipped both to capture the complex interactions among these dynamic signals and to strike an effective balance between immediate scheduling gain and long-horizon throughput.

In summary, effective global routing for LLM Agent serving demands a *coordinated* design of queuing and dispatch: the queuing layer must reflect workflow progress and accumulated urgency without relying on brittle static topology assumptions, while the dispatch layer must make adaptive decisions over high-dimensional, time-varying cluster state. These requirements motivate **RAGS** (Section 3), which pairs the DynA topology-agnostic queuing policy with an RL-based dispatcher.

3 Algorithm Design

3.1 System Architecture

Figure 4 illustrates the overall system architecture and request processing pipeline of RAGS. The system frontend accepts heterogeneous Agent workflows submitted by users and routes all LLM inference requests to the global scheduler for unified management. Within the scheduler, requests first enter the global request queue, where the Dynamic-Aware Queuing algorithm DynA computes scheduling priorities and reorders them accordingly. Subsequently, the RL-based scheduling algorithm asynchronously dispatches requests to target instances by combining queue characteristics with real-time cluster state. After inference completes, the system automatically triggers subsequent workflow stages, while underlying instances continuously report real-time load to the scheduler, forming a closed-loop state update. Upon full workflow completion, the final result is immediately returned to the user.

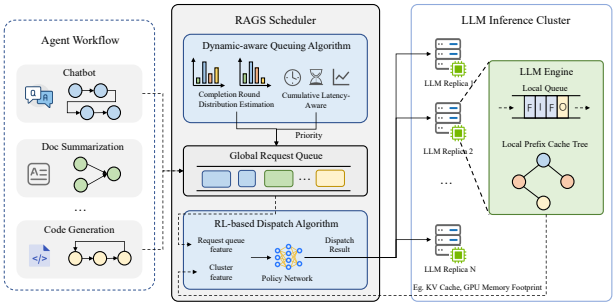


Figure 4: RAGS system architecture.

Based on this architecture, we elaborate on the two core components—the queuing algorithm and the dispatch algorithm—in the following subsections.

3.2 DynA: Dynamic-Aware Queuing Algorithm

3.2.1 Overview. Due to dynamic branching and autoregressive uncertainty in Agent workflows, traditional queue policies based on static topology are fundamentally limited: they assume the workflow structure is known in advance, an assumption that is often unmet in practice.

To address this, we propose **DynA** (Dynamic-Aware Queuing). Its core idea is to make all queuing decisions using only runtime-observable signals, without relying on any predefined workflow topology. Specifically, DynA leverages three types of signals:

(1) **Completion round distributions**: By continuously tracking the historical completion rounds for each Agent type, the system builds a probabilistic distribution of estimated remaining steps—without needing to know the specific workflow topology.
(2) **Cumulative latency**: The end-to-end latency accumulated by each request since its workflow was initiated, used as a real-time measure of urgency.
(3) **Request queuing time**: The waiting time each request has spent in the global scheduler queue.

DynA operates at two levels: (1) *inter-Agent* priority ordering based on workflow progress awareness, and (2) *intra-Agent* request ordering based on cumulative latency awareness.

3.2.2 Workflow-Progress-Aware Inter-Agent Priority. To differentiate the execution progress of requests from different Agent types—and thus prioritize requests in the later stages of their workflows—without knowing the workflow topology, DynA introduces an *online completion round distribution* mechanism.

Let $F_a(n)$ denote the empirical CDF of completion rounds for Agent type a, i.e., the probability that the total number of LLM calls in any completed workflow instance of this type does not exceed n. This distribution is maintained at runtime by continuously collecting statistics from completed workflows, without any prior assumption about static workflow topology.

Specifically, each time an Agent workflow of type a completes, the system records the completion round count n_j and updates $F_a(n)$ via Exponential Moving Average (EMA):

$$F_a^{(t+1)}(n) = (1-\lambda)\cdot F_a^{(t)}(n) + \lambda \cdot \mathbb{1}[n_j \le n], \tag{1}$$

where $\lambda \in (0,1)$ is the smoothing coefficient and $\mathbb{1}[\cdot]$ is the indicator function. By assigning higher weight to recent observations, EMA enables $F_a(n)$ to adapt to dynamic workload changes while filtering out stale historical data.

For Agent types with no historical records, the system initializes F_a with a uniform prior on $[1, N_{\max}]$:

$$F_a(n) = \begin{cases} 0, & n < 1 \\ n/N_{\max}, & 1 \le n \le N_{\max} \\ 1, & n > N_{\max} \end{cases}.$$

During online operation, EMA drives this prior toward the true empirical distribution.

Based on this real-time empirical distribution, when the scheduler receives a request from Agent type a that has already completed k LLM calls, it computes the *conditional near-completion probability* $\hat{P}_{a,k}$—the probability of reaching the terminal state within the next Δ calls given that k calls have already been made:

$$\hat{P}_{a,k} = P(k \le N_a \le k+\Delta \mid N_a \ge k) = \frac{F_a(k+\Delta) - F_a(k-1)}{1 - F_a(k-1)}, \tag{2}$$

where N_a is the random variable representing total LLM calls for Agent type a, and Δ is a look-ahead window size (default Δ=1). A larger $\hat{P}_{a,k}$ indicates a higher probability of near-term completion,

so DynA assigns higher scheduling priority to requests with larger $\hat{P}_{a,k}$. This design borrows from the Shortest Job First (SJF) principle while avoiding any dependence on a priori topology.

3.2.3 Cumulative-Latency-Aware Intra-Agent Ordering. For concurrent requests of the same type and same round, DynA applies an intra-Agent ordering strategy based on workflow start time. Let t_r^{start} be the initial trigger time of the workflow containing request r; the system prioritizes requests with earlier t_r^{start}, compensating requests that have already accumulated longer delays.

To prevent starvation of long-horizon Agent types under this priority ordering, DynA introduces a queue-age-aware anti-starvation mechanism. Define the current queuing wait time of request r as $\tau_r = t_{\text{current}} - t_r^{\text{arrive}}$. When τ_r reaches a system-defined maximum tolerance threshold $\tau_{\max}$, the following priority boost is triggered:

$$\text{Boost}(r) = \max\left(0, \frac{\tau_r - \tau_{\max}}{\tau_{\max}}\right). \tag{3}$$

Combining the conditional near-completion probability with the anti-starvation mechanism, DynA computes a comprehensive priority score for each queued request r:

$$\text{Score}(r) = \hat{P}_{a_r,k_r} + \eta \cdot \text{Boost}(r), \tag{4}$$

where a_r is the Agent type of r, k_r is the total LLM call count so far for the workflow instance, and η is the anti-starvation weight hyperparameter. Requests with higher scores are scheduled first; ties are broken by Agent start time. The complete DynA policy is given in Algorithm 1.

Algorithm 1 Dynamic-Aware Queuing Algorithm (DynA)

Require: Request queue Q; online completion-round distributions $\{F_a\}$; anti-starvation threshold $\tau_{\max}$; weight η
Ensure: Next request to dispatch r^*

```
for each request r ∈ Q do
    Retrieve Agent type a_r and completed call count k_r
    if F_{a_r} is not yet initialized then
        F_{a_r}(n) ← min(n, N_max)/N_max    // uniform prior
    end if
    P̂_{a_r,k_r} ← (F_{a_r}(k_r+Δ) − F_{a_r}(k_r−1)) / (1 − F_{a_r}(k_r−1))
    τ_r ← t_current − t_r^arrive
    Boost(r) ← max(0, (τ_r − τ_max)/τ_max)
    Score(r) ← P̂_{a_r,k_r} + η · Boost(r)
end for
Sort all requests in Q by (Score(r) desc, t_r^start asc)
r* ← first request in Q
return r*
```

3.3 RL-based Dispatch Algorithm

3.3.1 Problem Formulation and MDP Definition. We formalize the global scheduler's dispatch process as a Markov Decision Process (MDP) and solve for the optimal dispatch policy using reinforcement learning.

Consider a service system with K homogeneous LLM instances, each equipped with GPU G and deploying model M. During system operation, N Agent workflows arrive online over time. At any decision step t, let Q_t denote the set of pending requests in the global scheduler. The scheduler assigns a target LLM instance a_t to each request $q_{i:k}$ (the k-th call of workflow w_i). The optimization objective is to minimize the average end-to-end latency across all N workflows:

$$\min_{a_t} \frac{1}{N} \sum_{i=1}^{N} l_i, \tag{5}$$

where l_i is the actual end-to-end completion time of w_i.

Since Q_t may contain multiple concurrent requests, a centralized joint assignment would require an action space of $O(|Q_t| \times K)$. To avoid the curse of dimensionality, we adopt a progressive decision paradigm: the agent assigns one request at a time to a target instance.

The MDP is defined as follows. Every Δt, the global scheduler determines target LLM instances for all requests in the waiting queue. At step t, the agent observes state $s_{t:j}$ and outputs a discrete action $a_t^j \in \{1, \ldots, K\}$, routing the j-th queued request r_j to the a_t^j-th LLM instance.

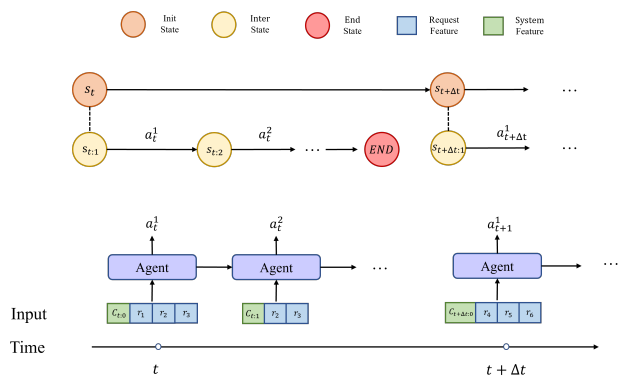


Figure 5: Markov Decision Process for global dispatch.

3.3.2 State Space. The state s_t encodes the current global system state, comprising features of the pending request set and the load status of each LLM instance.

Request state. The matrix $\mathbf{X}_t^{req}$ comprises four distinct feature vectors:

- $\vec{p}$ is the input length vector, where p_i denotes the number of input tokens for the i-th queued request.
- $\vec{d}$ is the estimated output length vector, where d_i represents the predicted output length of the i-th request.
- $\vec{u}$ is the parallel-stage indicator vector, where $u_i \in \{0, 1\}$ flags whether the i-th request resides in a parallelizable workflow stage.
- $\vec{\kappa}$ is the workflow type vector, where κ_i uniquely identifies the Agent application class of the i-th request.

Regarding the estimated output length d_i, we follow the methodology in [12] by predicting discrete length intervals rather than exact token counts. This coarse-grained approximation proves sufficient, as our designed MDP relies solely on relative interval magnitudes. Empirical evaluations demonstrate an interval prediction accuracy of approximately 70%.

Cluster state. The matrix $\mathbf{X}_t^{inst}$ is constructed by concatenating individual per-instance feature vectors. Specifically, the runtime profile of each LLM instance is characterized by:

- $\vec{m}$ and $\vec{g}$: One-hot encoded vectors specifying the instance's deployed model architecture and underlying GPU hardware, respectively.
- $\vec{wq}$: A load histogram of the waiting queue, derived by binning input lengths with a configurable step size *bin_width*.
- $\vec{rq}$: A load histogram of active execution queues. Prefill-stage requests are binned by input token length, whereas Decode-stage requests are binned by estimated output length.
- c: GPU memory utilization, computed as the proportion of allocated KV-cache blocks to the total block capacity.
- s: The expected prefix-cache reuse rate for the pending request, quantified as the fraction of reusable tokens.
- b: The estimated batch execution latency, maintained via a sliding-window average of recent iteration times.

To ensure training stability, all features are strictly normalized. The comprehensive MDP state representation is thus formulated as $s_t = [\mathbf{X}_t^{req} \,\|\, \mathbf{X}_t^{inst}]$, where $\|$ denotes the concatenation operator.

3.3.3 Reward Function. Using the end-to-end workflow latency directly as a reward signal would cause severe reward sparsity and credit-assignment problems due to the asynchronous execution model of LLM inference systems. Instead, we design a *composite immediate reward* that provides feedback after every dispatch decision:

$$r_t = \lambda_k \cdot r_k + \lambda_{imb} \cdot r_{imb} + \lambda_p \cdot r_p + \lambda_T \cdot T(x_{t,i}^{inst}), \tag{6}$$

where the terms are:

- r_k: *Prefix cache reuse reward*—expected KV-cache reuse rate on the target instance, encouraging lower time-to-first-token.
- r_{imb}: *Load-imbalance penalty*—measures the impact of the current dispatch on global load uniformity.
- r_p: *Queue penalty*—current waiting-queue length of the target instance, discouraging dispatch to overloaded nodes.
- $T(x_{t,i}^{inst})$: *Throughput estimate*—expected token throughput of the target instance in its current state.

To quantify load imbalance r_{imb}, the total load of instance i is estimated from its active request distribution ($\vec{rq}$) and waiting queue distribution ($\vec{wq}$):

$$L_i = \sum_{j=1}^{N_{bin}} (C_j \cdot N_{run,j}) + \sum_{j=1}^{N_{bin}} (C_j \cdot N_{pend,j}), \tag{7}$$

where C_j is the center length of bin j and $N_{\cdot,j}$ is the count of requests in that bin. The normalized imbalance score is:

$$S_{imb} = \frac{\sum_{i=1}^{K} |L_i - \bar{L}|}{\sum_{i=1}^{K} L_i + \epsilon}, \tag{8}$$

with $\bar{L} = \frac{1}{K}\sum_i L_i$ and ϵ a small constant for numerical stability. We set $r_{imb} = -S_{imb}$.

For the throughput estimate $T(x_{t,i}^{inst})$, we train a lightweight neural network surrogate offline on tens of thousands of real scheduling trajectories using MSE loss, achieving approximately 10% prediction error.

3.3.4 Policy Network Architecture. To capture complex temporal dependencies and heterogeneous instance features, we design a *Transformer-based scheduling network* that jointly encodes the current target request, the waiting-queue context, and the real-time state of each instance. Figure 6 shows the network topology.

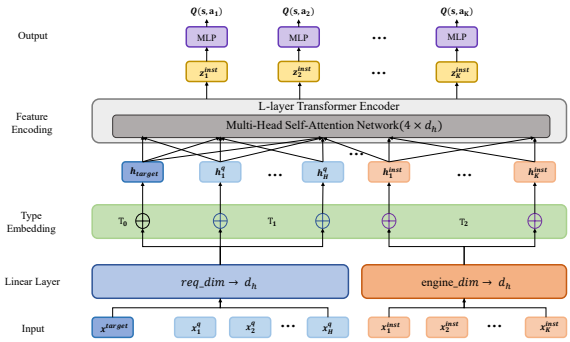


Figure 6: Dispatch policy network architecture.

The input layer is divided into three feature subspaces: the current target request $\mathbf{x}^{target}$, a look-ahead waiting queue $\{\mathbf{x}_i^q\}_{i=1}^H$ of length H, and the real-time states of K LLM instances $\{\mathbf{x}_j^{inst}\}_{j=1}^K$.

A linear layer first projects all features to a unified hidden dimension d_h:

$$\mathbf{e} = \text{Linear}(\mathbf{x}), \quad \mathbf{e} \in \mathbb{R}^{d_h}. \tag{9}$$

To help the Transformer distinguish different token roles, we add learned type embeddings:

$$\mathbf{h}_i = \mathbf{e}_i + \text{Embedding}(\text{type}_i), \quad \text{type}_i \in \{0{:}\text{Target},\ 1{:}\text{Queue},\ 2{:}\text{Inst}\}. \tag{10}$$

The combined sequence $\mathbf{S} = [\mathbf{h}_{target}, \mathbf{h}_{q,1}, \ldots, \mathbf{h}_{q,H}, \mathbf{h}_{inst,1}, \ldots, \mathbf{h}_{inst,K}]$ is passed through L Transformer encoder layers:

$$\mathbf{S}^{(l+1)} = \text{TransformerEncoder}\left(\mathbf{S}^{(l)}\right). \tag{11}$$

At the output layer, we extract the K instance-token representations $\{\mathbf{z}_{inst,j}\}_{j=1}^K$ and map each to an action-value score via a shared-parameter Q-value head:

$$Q(s, a_j) = \mathbf{W}_Q \cdot \text{ReLU}\left(\mathbf{W}_H \mathbf{z}_{inst,j} + \mathbf{b}_H\right) + b_Q. \tag{12}$$

The final output $\mathbf{Q} \in \mathbb{R}^K$ gives the expected cumulative return for dispatching the current request to each instance.

3.3.5 Reinforcement Learning Training. We adopt a two-phase scheme combining offline pretraining and online fine-tuning.

Offline Pretraining. We construct a data-driven offline simulation environment by replaying real production workload traces covering diverse Agent workflows (e.g., multi-turn dialogue, code generation, document summarization) at various load intensities, improving generalization.

For the high-dimensional discrete action space, we adopt the *Double-DQN* algorithm [23], which decouples target action selection from value estimation to suppress Q-value overestimation. We replace the original MSE loss with Huber Loss [11], which applies linear penalty for large TD errors, significantly improving training stability under heterogeneous request sequences.

Online Fine-tuning. Although offline training establishes baseline scheduling capabilities, static policies often suffer performance degradation in highly dynamic production clusters. Consequently, we introduce an online fine-tuning mechanism. The agent continuously records interaction transitions into an experience replay buffer, triggering periodic parameter updates at a low learning rate once a predefined sample threshold is reached. To maintain Quality of Service stability, we bound the ϵ-greedy exploration rate and employ momentum-based soft updates for the target network, effectively mitigating Q-value oscillations under distribution shifts.

4 Implementation

To support RL agent training and system evaluation, we build our testbed upon Vidur [1], an open-source LLM inference simulator. Vidur natively supports heterogeneous GPUs (e.g., A100, H100) and provides built-in benchmarks for mainstream models like Llama and Qwen, enabling rapid validation of scheduling algorithms across diverse configurations.

However, native Vidur primarily targets single-turn LLM requests. It lacks the capability to simulate the multi-step interaction dependencies inherent in Agent workflows, nor does it support prefix-cache reuse. To address these limitations, we augment Vidur's core engine along two dimensions:

4.1 KV Cache Reuse

To enable KV cache reuse within the LLM engine (i.e., the local scheduler), we implement a `tree_cache` module atop Vidur's native `memory_pool`. This module comprises two core components: a *Block Manager* responsible for the allocation and reclamation of memory blocks, and a *Local Prefix Cache Tree* that maintains the prefix-mapping relationships of KV caches within each LLM instance.

As shown in Figure 7, different request prefixes branch from the root node. Each node contains three types of information:

- **Token sequence**: the sequence of token IDs whose KV cache is stored at this node;
- **Reference count** (`ref`): the number of active requests currently sharing this prefix, used as the basis for memory reclamation;
- **Block index** (`block`): the memory block number in the Block Manager where the actual KV vectors are stored, establishing a mapping between tokens and memory blocks.

To support PagedAttention, the Local Prefix Cache Tree adopts a block-granularity management strategy: the system enforces block alignment via a `page_size` parameter, ensuring that only complete blocks are treated as valid cache entries. This design avoids the complex memory-slicing issues caused by unaligned token matching, keeping cache logic consistent with the underlying paged memory management.

In addition, the global scheduler maintains a *global KV cache prefix tree* to record the cache states across all LLM instances. Its logic mirrors the local tree, with the sole difference that each node in the global tree stores a mapping from tree nodes to LLM instances (`replica`). When a user request arrives, the global scheduler can quickly locate matching instances through this global tree, enabling cross-instance KV cache reuse.

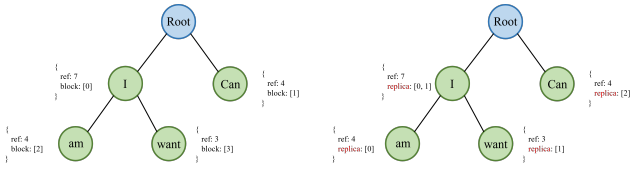


Figure 7: KV cache prefix trees: local (left) and global (right).

4.2 Agent Workflow Support

Since Vidur cannot directly simulate complex Agent workflow logic, we extend its underlying implementation. Specifically, we design and implement a `UnifiedRequest` object on top of `Request`. This object retains base request attributes via inheritance and further extends them with `agent_type` and `agent_id` metadata to identify the category and unique identity of a specific Agent workflow.

`UnifiedRequest` also introduces a *workflow graph* to capture the call-chain dependencies of the Agent. During simulation execution, the system tracks request completion states in real time: only after all preceding dependency requests in the workflow graph have completed and returned results does the system activate and emit the subsequent stage requests of the Agent workflow. Through this precise control of serial and parallel dependencies, we achieve faithful simulation of complex multi-step Agent workflow interactions.

5 Evaluation

5.1 Experimental Setup

5.1.1 Hardware and Software Environment. All model training and experimental evaluation are conducted on a physical server running Ubuntu 18.04.6 LTS, equipped with dual Intel Xeon E5-2680 v3 processors (2.50 GHz) and two NVIDIA GeForce RTX 3090 GPUs.

For inference simulation we use the Vidur simulator. We emulate a four-node LLM inference cluster in which each node is equipped with a 40 GB NVIDIA A100 GPU and independently deploys one Meta-Llama-3-8B instance. The prefill chunk size (`chunk_size`) is set to 4096 and the maximum batch capacity (`batch_size`) to 512.

Agent workflows are implemented with the LangChain framework [5] to generate realistic request sequences and workflow topologies for the experiments.

5.1.2 RL Training Configuration. We train the scheduling policy with Double-DQN using the following hyperparameters. The learning rate is 5×10^{-5}, the discount factor $\gamma = 0.995$, and the experience replay buffer holds up to 50 000 samples with a minimum warm-up of 500 samples before training starts; each gradient update uses a mini-batch of 256 samples. The target network is updated every 100 steps, model weights are saved every 100 steps, and training runs for 10 epochs. The policy network hidden dimension is 512. The ε-greedy exploration rate is initialized at 0.5 and decays to 0.01.

For DynA, the look-ahead window $\Delta = 1$, the anti-starvation age threshold $\tau_{\max} = 10.0$ s, and the weight $\eta = 0.1$.

5.1.3 Agent Workloads. To comprehensively evaluate the proposed method across different Agent workflow types, we select three representative LLM Agent workloads covering serial, parallel, and hybrid topologies. Figure 8 shows the token-length and request-count distributions of each workload.

ChatBot (serial). We build approximately 8 000 multi-turn dialogue workflows from the ShareGPT dataset [8], each containing 2–20 dialogue rounds. The input length follows a right-skewed distribution (mean ≈837.54 tokens, concentrated in 0–2000 tokens), while output lengths are more concentrated (mean ≈191.78 tokens), reflecting the long-input/short-output pattern of conversation tasks.

Map-Reduce Summarization (parallel). We construct a Map-Reduce document summarization Agent: the Map phase splits long documents into chunks and issues concurrent LLM requests for sub-summaries; the Reduce phase aggregates them into a final output. Using the Wikidata dataset [24], we build approximately 1 000 workflows with 2–50 LLM requests each. The mean input length is ≈833.51 tokens with higher variance than ChatBot, and the mean output length is ≈186.51 tokens.

Code Generation Agent (hybrid). We design a multi-stage Agent following the pipeline User → Planner → Workers → Evaluators → Verifier → Final Output. If the code fails evaluation, a feedback loop triggers the Planner to re-schedule Workers for iterative repair. Using HumanEval [7] and MBPP [4], we build ≈1 000 workflows with 2–50 LLM requests each. The mean input length is ≈3541.76 tokens (significantly higher than the other two workloads, due to the feedback repair context), and the mean output length is ≈800.87 tokens.

Evaluation proceeds in two steps: first, each of the three workloads is tested in isolation; then a mixed workload combining all three types in preset ratios is evaluated. Workflow inter-arrival times follow a Poisson distribution. All experiments use the Llama3-8B model by default.

5.2 Baselines

We compare RAGS against the following baselines.

Round-Robin. Serving as our naive baseline, this strategy pairs First-Come-First-Serve (FCFS) queuing with round-robin dispatch. It treats all inference requests independently, operating entirely without Agent workflow awareness.

Parrot [14]. A state-of-the-art end-to-end LLM application serving system. Parrot utilizes semantic variable abstraction to expose inter-request DAG dependencies and prompt structures to the underlying serving layer. At the scheduling level, it aggregates similar requests to satisfy application-level performance goals; at the dispatch level, it employs session affinity to route correlated requests, thereby maximizing KV-cache prefix reuse. In our evaluation, we faithfully replicate its core FCFS queuing and session affinity routing algorithms, abstracting away hardware-specific engineering optimizations.

Ayo [22]. A fine-grained LLM orchestration framework that translates workflows into task-level dataflow graphs, leveraging dependency pruning and prefill splitting. Its queuing policy prioritizes requests based on their topological depth, while its dispatch mechanism relies on basic round-robin routing. Similar to Parrot, we isolate and implement only its topology-aware queuing and routing logic for comparison.

5.3 Main Results

Figure 9 shows the end-to-end performance of each method under the three single-type workloads at varying request arrival rates.

ChatBot (serial). At an arrival rate of 20 workflows/s, RAGS achieves up to **5.2**× lower mean E2E latency than Round-Robin and Ayo. For TTFT, Round-Robin's P50 surges to 17 s under high concurrency, while RAGS keeps it below 1 s through RL-based dynamic load balancing and DynA's priority queuing. RAGS's speedup over Parrot is only ≈1.1× in this scenario because ChatBot has a fixed topology with highly regular KV-cache sharing, allowing Parrot's static prefix-aware routing to effectively exploit the cache; the headroom for dynamic routing is therefore limited.

Map-Reduce Summarization (parallel). Since this workflow has a simple two-level topology, the overall gains of RAGS over baselines are moderate. At 20 workflows/s, RAGS reduces mean E2E latency by only ≈6% over Round-Robin. Ayo's topology-based priority essentially degenerates to round-robin because it cannot distinguish among the many same-level Map nodes. DynA, however, estimates workflow progress and preferentially schedules requests about to trigger the Reduce aggregation, partially alleviating inter-node synchronization blocking.

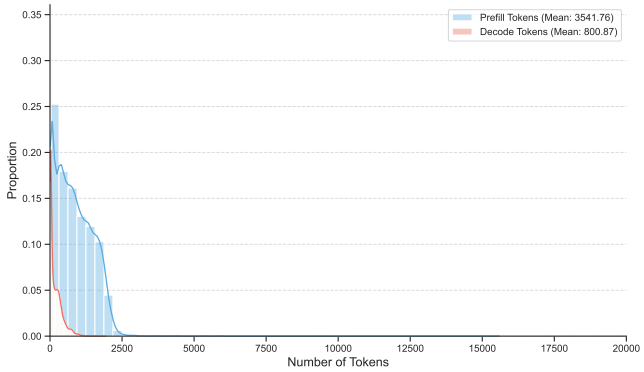
Code Generation Agent (hybrid). This workload introduces autonomous feedback-repair loops and is designed to assess the scheduler's ability to handle highly dynamic topologies. Parrot causes severe queue congestion in this scenario: its mean E2E latency reaches 240 s and P95 tail latency 600 s— approximately 3–4× higher than RAGS. The root cause is that the repair feedback loops create highly dynamic per-session compute loads, and static session affinity triggers severe global load imbalance, causing head-of-line blocking on overloaded instances. Thanks to the synergy between RL-based dynamic load balancing and DynA's preemptive scheduling of high-priority requests, RAGS stabilizes mean latency in the range of 70–85 s across most load levels, and maintains P95 tail latency around 200 s even at 5 workflows/s.

TTFT further reveals the degree of queue congestion under highly dynamic load. Parrot's P50 TTFT spikes to ≈6 s at 5 workflows/s. RAGS, by jointly optimizing KV-cache reuse and load balance, keeps P50 TTFT below 0.5 s under the same heavy load, demonstrating that two-tier co-design effectively suppresses TTFT spikes in dynamic workflows.

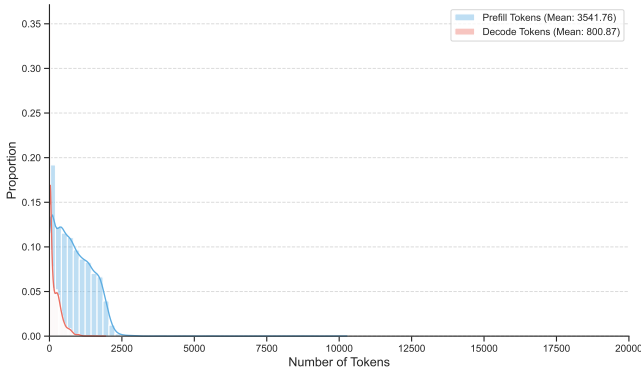
Mixed workload. Real production environments feature highly heterogeneous mixed workloads. We construct a mixed dataset of 500 workflows and vary the proportions (x, y, z) of ChatBot, Summarization, and Code Agent workflows. The arrival rate is fixed at 5 workflows/s. Figure 10 presents mean E2E latency under four representative mixture ratios.

When the mix contains no Code Agent tasks (ratio $(0.5, 0.5, 0)$), most baselines converge to 11–14 s, with Parrot achieving 11.4 s (second best) and RAGS achieving 10.6 s (best). Once Code Agent tasks with complex feedback loops are introduced (e.g., ratios $(0, 0.5, 0.5)$ and $(0.5, 0, 0.5)$), Parrot's latency spikes to ≈40 s, exposing the inability of static binding to handle high compute density. Ayo and Round-Robin show similar degradation, confirming that static topology-depth sorting cannot cope with heterogeneous concurrent traffic. RAGS achieves 18.7 s under the balanced mix $(0.33, 0.34, 0.33)$—14.5% lower than Ayo and 28.6% lower than Parrot—and keeps mean E2E latency below 22.8 s in every Code-Agent-inclusive scenario.

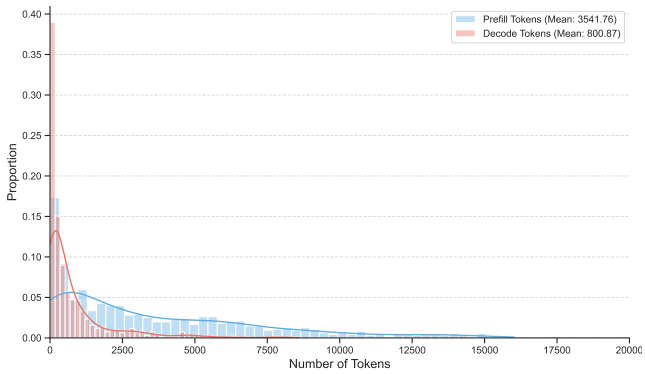
5.4 Ablation Studies



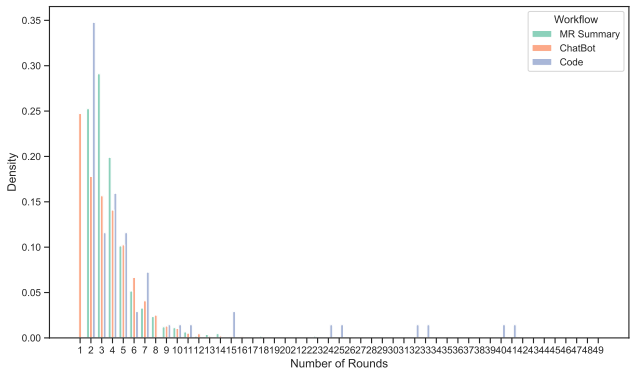
(a) Token Distribution of ShareGPT Dataset



(b) Token Distribution of Wikidata Dataset



(c) Token Distribution of CodeAgent Dataset



(d) Request Count Distribution of Different Workflows

Figure 8: Feature Distributions of Different Agent Workflows

Figure 9: End-to-end performance comparison under three single-type workloads at varying request rates.

5.4.1 Effect of State-Space Bin Width. Figure 11 shows the impact of the bin width *bin_width* used in the load histogram on mean E2E latency. Too small a bin width leads to sparse features that increase RL convergence difficulty; too large a bin width obscures load differences between instances. A bin width of 64 achieves the best balance between representation fidelity and learning efficiency and is used as the default.

5.4.2 Component Ablation. To quantify the individual contributions of the queuing and dispatch components, we run an ablation on the mixed workload (ChatBot:Summarization:Code = 0.33:0.34:0.33) at varying arrival rates, comparing three configurations: (1) **RAGS**: full system (DynA + RL dispatch); (2) **RAGS w/o DynA**: FCFS queuing + RL dispatch; (3) **RAGS w/o RL**: DynA queuing + Round-Robin dispatch.

As shown in Figure 12, performance differences are small at low arrival rates (RAGS: 11.206 s, RAGS w/o DynA: 11.205 s, RAGS w/o RL: 12.867 s) but widen significantly at high loads. At the peak arrival rate, RAGS achieves 104.557 s; removing DynA raises latency to 110.804 s; removing RL dispatch degrades latency further to 148.758 s.

These results lead to three conclusions. First, RL dispatch is the dominant contributor: removing it increases peak mean latency by ≈42.27%, confirming that its benefit scales with system load. Second, DynA also provides non-trivial gains: at a medium arrival rate RAGS w/o DynA's latency is ≈14.8% higher than RAGS (20.782 s vs. 18.104 s), showing that workflow-progress-aware priority ordering

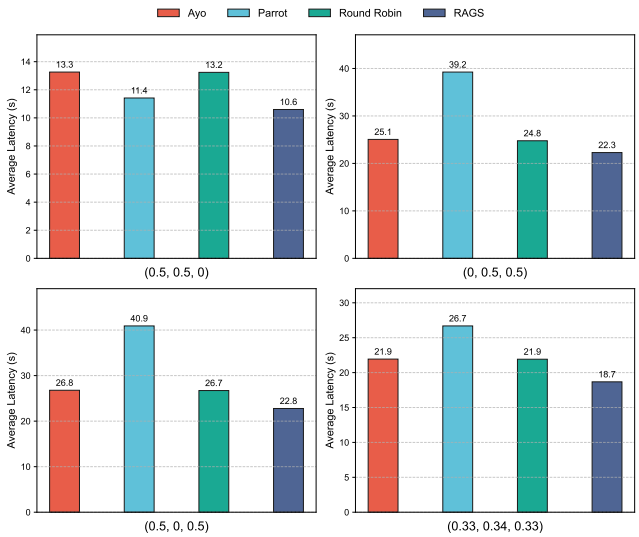


Figure 10: End-to-end performance comparison under mixed loads.

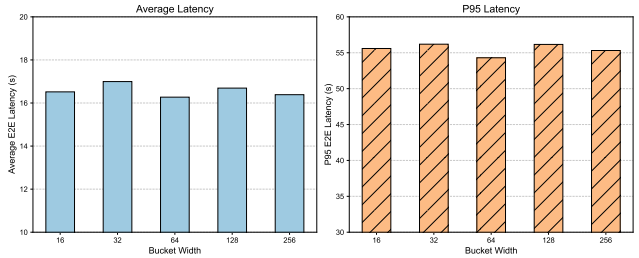


Figure 11: Effect of bin width on mean E2E latency.

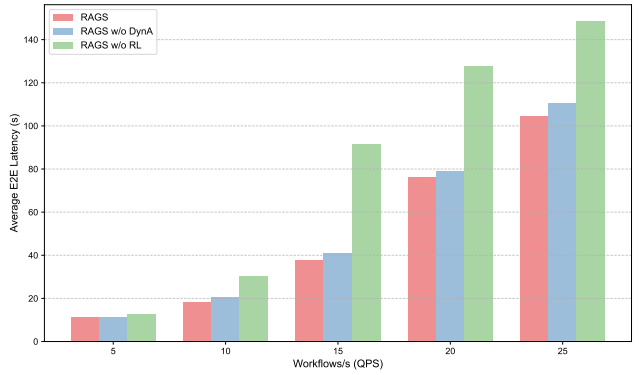


Figure 12: Component ablation under mixed workload.

effectively reduces head-of-line blocking. Third, the two components are complementary: DynA ensures that critical requests enter the dispatch stage first, while RL global load balancing prevents those requests from encountering secondary queuing inside LLM engines.

5.5 Heterogeneous Cluster Evaluation

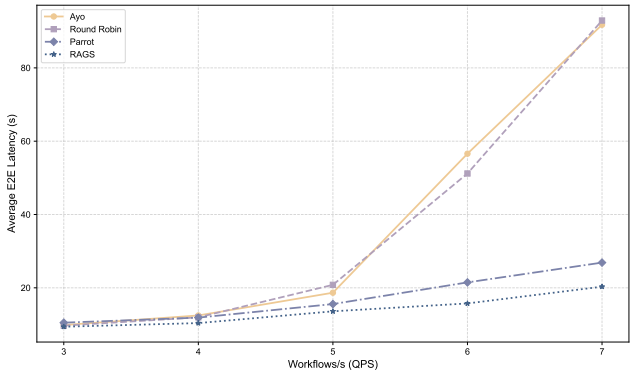


Figure 13: Heterogeneous GPU cluster evaluation results.

To validate RAGS on a heterogeneous GPU cluster, we run experiments on a 4-GPU cluster with 2 NVIDIA A100s and 2 NVIDIA H100s. Since complete performance benchmarks for Meta-Llama-3-8B on H100 are not yet available, all instances deploy meta-llama/Llama-2-7b-hf. A100 and H100 differ significantly in compute throughput and memory bandwidth, so the scheduler must sense hardware heterogeneity while avoiding resource waste from unbalanced allocation.

As shown in Figure 13, all methods exhibit increasing mean E2E latency as the arrival rate rises from 3 to 7 workflows/s, but with markedly different slopes. Round-Robin ignores GPU heterogeneity and distributes evenly, turning the weaker A100 instances into system bottlenecks; its latency surges above 80 s at high concurrency. Parrot's static session-binding also lacks hardware-awareness: routing computationally intensive requests to weaker A100 instances easily triggers severe local overload.

RAGS maintains the lowest mean E2E latency at all load levels. Its advantage stems from the joint encoding of GPU type and instance load in the RL state space, which allows the policy to implicitly sense inter-instance compute differences and adaptively balance hardware performance with global load distribution. At the peak arrival rate of 7 workflows/s, RAGS reduces mean E2E latency by approximately 30%–40% compared with Round-Robin and Parrot, validating its effectiveness and generalizability in heterogeneous inference clusters.

6 Related Work

LLM Inference Engines. vLLM [13] introduces PagedAttention to manage KV cache with OS-style virtual memory paging, greatly reducing memory fragmentation and improving GPU utilization. SGLang [26] further proposes RadixAttention, indexing all historical KV caches in a radix tree to enable fine-grained cross-request prefix reuse and cache-aware routing. Sarathi-Serve [2] mitigates prefill–decode interference via chunked prefills, while Orca [25] pioneered continuous batching to keep GPUs fully occupied. These systems optimize single-turn request serving but lack mechanisms to model multi-step workflow dependencies inherent in agent workloads.

LLM Agent Serving Systems. Parrot [14] is among the first to address agent workflow scheduling: it introduces *semantic variables* to expose inter-request data-flow to the serving layer and adopts session-affinity routing to maximize KV cache reuse across correlated requests. Ayo [22] decomposes complex LLM applications into primitive-level dataflow graphs and co-schedules LLM and non-LLM stages in a pipeline, cutting end-to-end latency. Autellix [15] elevates the full agent program to the scheduling unit and proposes PLAS/ATLAS—multi-level feedback queue schedulers that use accumulated execution time on the critical path to mitigate head-of-line blocking. KVFlow [18] focuses on cache management for multi-agent workflows: it builds an agent execution graph, assigns priority to cache nodes based on estimated reuse distance, and employs asynchronous prefetching together with a future-reuse-aware eviction policy to raise cache hit rates. HexGen-Flow [20] proposes a two-level SLO-aware scheduler for Text-to-SQL pipelines, decomposing end-to-end budgets proportionally and using urgency-based preemption; however, it is tightly coupled to a specific pipeline structure and does not generalize to arbitrary agent topologies.

RL for Cluster Scheduling. DeepRM [16] and DL2 [19] demonstrate the effectiveness of deep RL for resource scheduling in DNN training clusters. TORTA [10] applies RL to LLM inference traffic routing across regions, dynamically adjusting per-window routing ratios. Unlike these works, RAGS applies RL to *agent workflow* routing, explicitly encoding workflow-level dependency states and KV cache residency to drive adaptive global dispatching.

References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6 (2024), 351–366.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI)*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, Santa Clara, CA, 117–134.

[3] Amazon Web Services. 2026. *aws-genai-llm-chatbot: A modular and comprehensive solution to deploy a Multi-LLM and Multi-RAG powered chatbot.* https://github.com/aws-samples/aws-genai-llm-chatbot

[4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021), 1–34.

[5] Harrison Chase. 2022. LangChain. https://github.com/langchain-ai/langchain. Accessed: 2026-03-13.

[6] Jinyuan Chen, Jiuchen Shi, Quan Chen, and Minyi Guo. 2025. Kairos: Low-latency multi-agent serving with shared llms and excessive loads in the public cloud. *arXiv preprint arXiv:2508.06948* (2025), 1–15.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021), 1–35.

[8] Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. 2023. *Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.* https://lmsys.org/blog/2023-03-30-vicuna/

[9] cognitivetech. 2026. *ollama-ebook-summary: LLM for Long Text Summary (Comprehensive Bulleted Notes).* https://github.com/cognitivetech/ollama-ebook-summary

[10] Chengze Du, Zhiwei Yu, Heng Xu, Haojie Wang, Jialong Li, et al. 2025. Temporal-aware gpu resource allocation for distributed llm inference via reinforcement learning. *arXiv preprint arXiv:2507.10259* (2025), 1–16.

[11] Peter J. Huber. 1992. Robust Estimation of a Location Parameter. In *Breakthroughs in Statistics: Methodology and Distribution*, Samuel Kotz and Norman L. Johnson (Eds.). Springer, New York, 492–518.

[12] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S^3: Increasing GPU Utilization during Generative Inference for Higher Throughput. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, December 10–16, 2023, New Orleans, LA, USA (NeurIPS)*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). Curran Associates, Inc., Red Hook, NY, USA, 18015–18027.

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles (SOSP)*, Jason Flinn and Margo I. Seltzer (Eds.). Association for Computing Machinery, Koblenz, Germany, 611–626.

[14] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, Santa Clara, CA, 929–945.

[15] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025), 1–17.

[16] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. 2016. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM workshop on hot topics in networks (HotNets)*, Bryan Ford and Alex C. Snoeren (Eds.). Association for Computing Machinery, Atlanta, GA, 50–56.

[17] OpenCode AI. 2026. *OpenCode: A powerful AI coding agent. Built for the terminal.* https://github.com/opencode-ai/opencode

[18] Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *arXiv preprint arXiv:2507.07400* (2025), 1–12.

[19] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, Chen Meng, and Wei Lin. 2021. DL2: A deep learning-driven scheduler for deep learning clusters. *IEEE Transactions on Parallel and Distributed Systems* 32, 8 (2021), 1947–1960.

[20] You Peng, Youhe Jiang, Wenqi Jiang, Chen Wang, and Binhang Yuan. 2025. HEXGEN-FLOW: Optimizing LLM Inference Request Scheduling for Agentic Text-to-SQL. *arXiv preprint arXiv:2505.05286* (2025), 1–21.

[21] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. 2024. Preble: Efficient distributed prompt scheduling for llm serving. *arXiv preprint arXiv:2407.00023* (2024), 1–23.

[22] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS)*, Adrian Sampson, Martha Kim, and Christopher J. Rossbach (Eds.). Association for Computing Machinery, Rotterdam, Netherlands, 1302–1316.

[23] Hado Van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)*, Dale Schuurmans and Michael P. Wellman (Eds.), Vol. 30. AAAI Press, Phoenix, AZ, 2094–2100.

[24] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[25] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI)*, Marcos K. Aguilera and Hakim Weatherspoon (Eds.). USENIX Association, Carlsbad, CA, 521–538.

[26] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, December 10–15, 2024, Vancouver, BC, Canada (NeurIPS)*, Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). Curran Associates, Inc., Red Hook, NY, USA, 62557–62583.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009